ECS 160 – Microservices

Instructor: Tapti Palit

Teaching Assistant: **Xingming Xu**

Agenda

- **Microservices**
- HTTP Protocol and REST API
- Spring Boot

Microservices

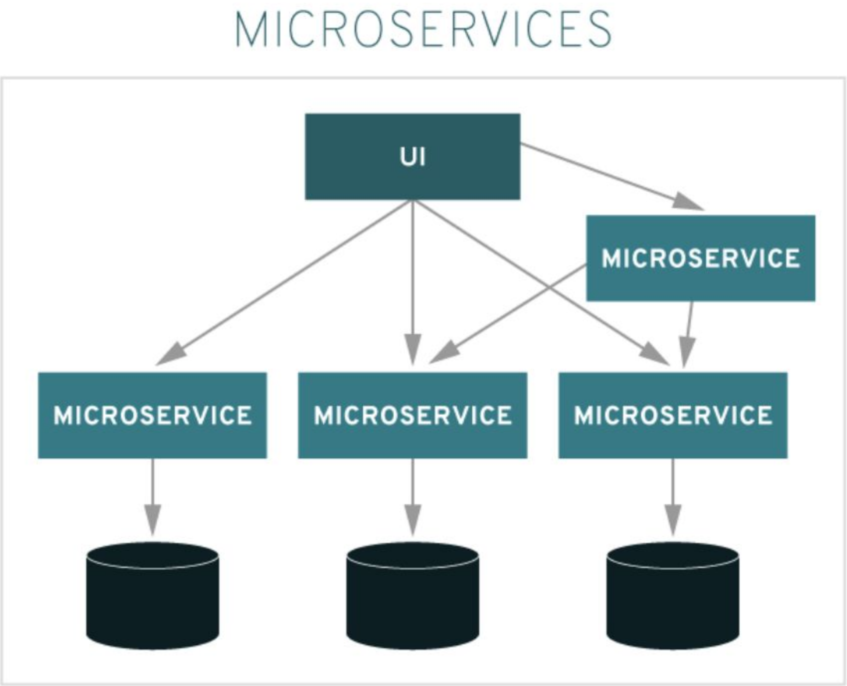
- Monolithic architecture vs microservices
  - Tightly-coupled vs decentralized
  - Processes as collections of independent components
- Each service is responsible for a single function

Benefits of Microservices

- Autonomy
- Specialized
- Scalable
- Deployable
- Resilient
- Reusable

Microservices as Architectural Style

- Independently deployable
- Loosely coupled
- Service-oriented
  - Implementation strategy



Source: https://www.redhat.com/en/topics/microservices/what-are-microservices

Organization

- Recall: loosely coupled…
  - Services should be organized around *specific* business capabilities
  - Serve specific functions and easily replaceable
- Requires supporting services to enable effective organization

Inter-service Communication

- Let’s talk protocols:
  - Agreed-upon ways for data to be structured, transmitted, validated
- We prefer lightweight, standard protocols
- Request-response model
  - Takes place within client-service architecture
  - Well-formed messages

Agenda

- Microservices
- **HTTP Protocol and REST API**
- Spring Boot

HTTP Protocol

- "Application level protocol" for remote communication
- Functions as request-response protocol:
    - Client (e.g. browser) sends requests, server (e.g. web server) responds with responses
- Server can provide the client with
    - HTML files
    - Content
    - Perform functions on behalf of client
    - etc.

Request Methods

- GET – requests information about state of resource state
- PUT – updates resource at **specified URI (unique resource identifier)**
  - Idempotent, multiple PUT requests to the same resource should have same effect on the resource
  - Wipes out any existing resource there completely, replaces with request body
- POST – creates new resource, sometimes performing actions
  - Non-idempotent, can affect the state of the server
  - Information carried in request body
- DELETE – deletes specific resource

Requests as Methods

- Each of these requests contains a “verb” defining a method
  - Enable APIs to perform CRUD (Create, Read, Update, Delete) actions in a standard and predictable way

REST

- REST **architectural style** is typically used to build web services and APIs
- Defined in 2000 <!>, seeks to defines how internet-scale architecture should be constructed and function
- Access **resources** at **endpoints**
- HTTP *protocol* implements these “RESTful principles”

Main REST Principles

- Uniform Interface – simplify architecture, improve interactions
- Client-server – separate concerns in client and server
- Stateless – client stores session state
    - Requests should contain all info necessary to understand request
- Cacheable – resources must label themselves cacheable
- Layered System – architecture composed of local layers
    - Components are restricted to operating within a layer, which represents a single set of concerns
- Code on demand – client can download code at runtime

Request Methods and REST

- Define endpoints (urls), and interact with them to make requests to servers
- We send requests with a method (“GET”, “POST”, etc.)

API Suggestions

- When creating endpoints, make their names meaningful
- Use **nouns** instead of verbs for resource names
- Use **plural** instead of singular names for collections
- Be consistent between returned structures
- Use JSON in requests and responses
- Avoid a large amount of small resources
- Don't mirror your database structure (and expose it to your client)

Source: https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design

Agenda

- Microservices
- HTTP Protocol and REST API
- **Spring Boot**

Creating Web Services with Spring Boot

- Spring Boot is a framework providing tools to build Spring-based applications
- Supports creating microservices, and makes development *much quicker*

Creating an API in Spring Boot

https://spring.io/guides/tutorials/rest